Introdução ao C++
Introdução à Análise de Complexidade

Estrutura de Dados

Universidade Federal de Juiz de Fora
Departamento de Ciência da Computação

Conteúdo

- Introdução à Linguagem C++
  - Tipos de dados
  - Entrada e Saída
  - Arquivos
  - Strings
- Introdução à Análise de Complexidade
  - Algoritmos recursivos
  - Exemplos

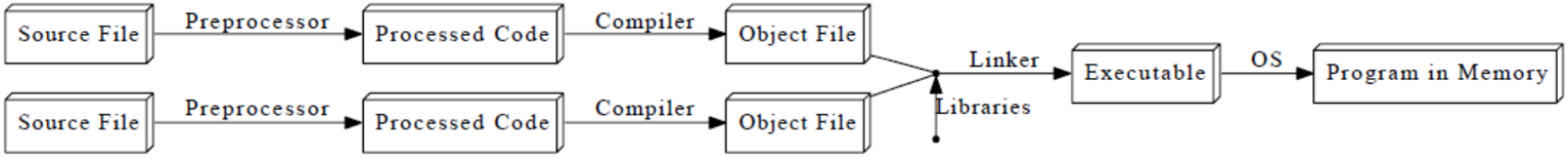
Introdução à Linguagem C++

Introdução

- A linguagem C++
- Criada em 1979
- Bjarne Stroustrup
- Altamente popular
- Linguagem compilada
- C++ estende a linguagem C: veremos que tudo que já aprendemos em C também é válido em C++
- Programação Orientada a Objetos

Introdução

- Ciclo de desenvolvimento



- Nas aulas de Laboratório de Programação II, o ambiente de desenvolvimento integrado Codeblocks será utilizado.
- Compilador: GNU GCC e GNU G++

Introdução

- Um primeiro programa em C++

```
#include <iostream>

using namespace std;

int main()
{
  cout << "Hello world!" << endl;
  return 0;
}
```

- Observações:
  - Biblioteca `iostream`
  - Saída em tela com operador `cout`
  - `namespace`

Tipos de Dados

- `char`: Caracter (1 byte)
- `int`: Inteiro (4 bytes)
- `float`: Número real em ponto flutuante (4 bytes)
- `double`: Número real em ponto flutuante (8 bytes)
- `bool`: Valores lógicos: true ou false
- `string`: Cadeia de caracteres

Tipos de Dados

- Exemplo

```
#include <iostream>
using namespace std;

int main ()
{
  int x;
  float y;
  x = 4 + 2;
  y = x/4;
  cout << x/3 << " " << x * 2 << endl;
  cout << " y = " << y << endl;
  return 0;
}
```

Tipos de Dados

bool

- Variáveis booleanas podem assumir apenas dois valores
  - `true` (1)
  - `false` (0)
- Para declarar variáveis booleanas usa-se a palavra chave `bool`.

```
int main()
{
  bool b1 = true;
  bool b2 = !b1;
  bool b3 = b1 && b2;

  cout << b1 << endl;
  cout << b2 << endl;
  cout << b3 << endl;

  return 0;
}
```

Tipos de Dados

Declaração de variáveis

- Em C++ pode-se declarar variáveis em qualquer região do código, assim como um outro comando qualquer.
- O escopo da variável começa no ponto onde ela é declarada e vai até o fim do bloco, isto é, até a chave } mais próxima.
- No exemplo abaixo, a variável `i` só existe dentro do `for`.

```
int main()
{
  int x, y, z, soma=0;
  // ...
  for(int i=0; i<10; i++)
  {
    soma = soma + x;
  }
  // ...
}
```

Operadores

Operador	**Significado**
$>$	Maior que
$>=$	Maior que ou igual a
$<$	Menor que
$<=$	Menor que ou igual a
$==$	Igual a
$!=$	Diferente de
&&	E
$\|\|$	Ou
!	Não

Entrada e Saída

- Em C++ para escrever algum texto na tela usamos o seguinte comando:

```
cout << "Mensagem";
```

- Podemos concatenar mais strings à saída usando várias vezes o operador <<:

```
cout << "X = " << x << "Y = " << y;
```

- Podemos inserir uma quebra de linha da seguinte forma

```
cout << "X = " << x << "Y = " << y << endl;
```

- Ou usando os formatadores especiais como em C

```
cout << "Hello World! \n";
```

- Note que para escrever texto na tela dessa forma é preciso incluir o seguinte comando no início do programa:

```
using namespace std;
```

Entrada e Saída

- A leitura de dados a partir do teclado é tão fácil quanto a escrita na tela.
- Não é preciso especificar o tipo de dados a ser lido, pois o operador `cin` converte a entrada para o tipo de dados da variável corretamente.
- Basta usar o operador `cin` como no exemplo abaixo:

```
#include <iostream>
using namespace std;
int main ()
{
  int i;
  cout << "Digite um inteiro: ";
  cin >> i;
  cout << "O valor digitado e " << i;
  cout << " e o seu dobro e " << i*2;
  cout << endl;
  return 0;
}
```

Entrada e Saída

```
int x, y;
cout << "Digite X: ";
cin >> x;
cout << "Digite Y: ";
cin >> y;

cout << "x*y = ";
cout << x*y << endl;

// ou .........

int x, y;
cout << "Digite X e Y:";
cin >> x >> y;

cout << "x*y = ";
cout << x*y << endl;
```

```
char sexo;
cout << "Sexo (m/f)?";
cin >> sexo;
if (sexo == 'm' )
  cout<<"Masculino"<<endl;
else
  cout<<"Feminino"<<endl;

int i;
float x, soma=0;
for (i=0; i<5; i++)
{
  cin >> x;
  soma = soma + x;
}
cout << "Soma = " << soma;
cout << endl;
```

Arquivos

- Ler e escrever conteúdo em arquivos de texto.
- É preciso incluir a biblioteca **`fstream`**

```
#include <fstream>
```

- Tipos de dados:
  - Arquivo de entrada: `ifstream`
  - Arquivo de saída: `ofstream`
- **Observação**: em C++ as bibliotecas nativas da linguagem a serem incluídas pelo usuário não terminam em .h
- Exemplos:

```
#include <iostream>
#include <cmath>
#include <fstream>
// etc ...
```

Arquivos

Saída

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
  int i, numAlunos=10;
  float notas[10];
  // ... obtem as notas ...
  ofstream arqSaida;
  arqSaida.open("saida.txt");                // abre
  arqSaida << "ESTRUTURA DE DADOS\n";
  for (i=0; i<numAlunos; i++){
    arqSaida << "Aluno " << i+1 << ": "; // escreve
    arqSaida << notas[i] << endl;
  }
  arqSaida.close();                          // fecha
  return 0;
}
```

Arquivos

Entrada

- Programa exemplo que lê de um arquivo texto (exemplo.txt) as seguintes informações:
  - Número de alunos (N)
  - E em seguida lê de cada uma das N linhas, o número de matrícula e a nota do aluno
- O programa deve ler as informações do arquivo e imprimir o seu conteúdo na tela.
- Exemplo de arquivo de entrada:

```
5
201199001 86
201199002 76
201199003 55
201199004 90
201199005 45
```

Arquivos

Entrada

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
  int i, numAlunos, mat, nota;
  ifstream arqEntrada;
  arqEntrada.open("exemplo.txt");        // abre
  arqEntrada >> numAlunos;               // le
  for (i=0; i<numAlunos; i++){
    arqEntrada >> mat >> nota;           // le
    cout << "Aluno " << i+1 << " : ";
    cout << nota << endl;
  }
  arqEntrada.close();                    // fecha
  return 0;
}
```

Strings

- Em C++ pode-se trabalhar com *strings* de duas formas:
  - No estilo da linguagem C
  - Com o tipo de dados `string` definido na biblioteca da linguagem
- Lembre-se que em C não existe um tipo de dados específico para isso:

```
char nome1[50];
char nome2[10] = "Fulano";
```

- As strings em C terminam com um caractere especial: \0

Strings

- É preciso incluir a biblioteca para usar strings.

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
  string str1;
  string str2 = "Bob Esponja";
  cout << "Digite um nome: ";
  cin >> str1;
  cout << "Personagem 1: " << str1 << endl;
  cout << "Personagem 2: " << str2 << endl;
  cout << "Terceira letra: " << str1[2];
  cout << endl;
  cout << "Primeira letra: " << str2[0];
  cout << endl;
  return 0;
}
```

Strings

- A grande diferença de trabalhar com o tipostring é que você declara uma variável deste tipo como uma simples variável, e não como um array (como é o caso da cadeia de caracteres em C).
- Exemplo: `char nome1[60];`
- Além disso, diversas operações com strings são mais simples quando usa-se o tipo de dados `string`.
  - Concatenação
  - Calcular o tamanho
  - Cópia
  - Procura caractere ou substring
  - Comparação, etc...

Strings

```
string str1 = "bom", str2 = " dia";
string str3, str;

str3 = str1 + str2;
str1 += str2;

cout << str1 << endl;
cout << str3 << endl;   // bom dia

str1 += " . boa noite";
cout << str1 << endl;   // bom dia . boa noite

int tam = str2.size();
cout << "Tamanho de str2 = ";
cout << tam << endl;   // Tamanho de str2 = 4

cin >> str;            // The Hobbit
cout << "A string digitada e: ";
cout << str << endl;   // The
```

Strings

- Para ler uma linha inteira, é preciso usar a função `getline(cin,str)`
- O primeiro argumento diz de onde pegar a entrada e o segundo argumento em qual string gravar a entrada.
- Exemplo:

```
string str;
getline(cin,str);

// Entrada: "The Hobbit"

cout << "A string digitada e: " << str;
cout << endl;

// Saida: A string digitada e: The Hobbit
```

Exercícios

(1) A definir.

Introdução à Análise de Complexidade

Análise de Complexidade

- Algoritmos demandam tempo de execução e recursos (memória, espaço em disco, dispositivos externos, etc).
- Analisar a alocação de recursos que um certo algoritmo demanda é importante na escolha de soluções mais rápidas ou que ocupem menos espaço de memória, por exemplo.
- Bons programadores se preocupam em implementar algoritmos que demandam o mínimo de recursos e executem no menor tempo possível.
- Embora um programa possa ser analisado sob vários aspectos, destaca-se a seguir a análise relativa ao seu desempenho, especialmente, em relação a medida do seu tempo de computação.

Análise de Complexidade

- A análise de complexidade de algoritmos é uma ferramenta que permite estudar como um algoritmo se comporta quando os dados de entrada aumentam.
- Se o algoritmo é alimentado com uma outra entrada, como o algoritmo se comporta?
- Se o algoritmo leva 1 segundo para executar para uma entrada de tamanho 1000, como ele irá se comportar se o tamanho da entrada for duplicado?

Análise de Complexidade

- Seja um programa com n instruções. Então, o tempo total de execução do programa T é dado por

$$T = \sum_{i=1}^{n} (t_i \, n_i)$$

- onde:
  - t_i é o tempo de execução da instrução i
  - n_i é o número de vezes que a instrução i é executada
- Entretanto, como o tempo de execução da instrução i é sempre de difícil obtenção, avalia-se o tempo total considerando somente o número de vezes que a instrução é executada.
- Esse número é chamado de contagem de freqüência ou simplesmente **freqüência da instrução** i.

Análise de Complexidade

- Exemplos de determinação da frequência f de uma instrução.
- Comando que pertence a uma sequência simples: tem frequência $f = 1$.

```
x = x + 1;
```

- Se essa instrução pertencer a uma estrutura de repetição

```
for(i=1; i<=n; i++)
{
  // ...
  x = x + 1;
  // ...
}
```

- Nesse caso a instrução tem frequência:

$$f = \sum_{i=1}^{n} 1 = n$$

Análise de Complexidade

- Se a estrutura do exemplo anterior pertencer a outra estrutura de repetição:

```
for(j=1; j<=n; j++)
{
  // ...
  for(i=1; i<=n; i++)
  {
    // ...
    x = x + 1;
    // ...
  }
  // ...
}
```

- Nesse caso a instrução tem frequência:

$$f = \sum_{j=1}^{n} \left(\sum_{i=1}^{n} 1 \right) = \sum_{j=1}^{n} n = n^2$$

Análise de Complexidade

- A maior frequência encontrada em um programa é chamada de **ordem de grandeza** de crescimento de tempo do programa.
- A ordem de grandeza de um algoritmo é o principal parâmetro para análise do desempenho de sua execução.
- Seja N um parâmetro que caracteriza o tamanho de um problema.
- As ordens de grandeza mais comuns nos algoritmos são:
  - $O(1) \rightarrow$ constante
  - $O(\log_2 N) \rightarrow$ logaritmo
  - $O(N) \rightarrow$ linear
  - $O(N \log_2 N)$
  - $O(N^2) \rightarrow$ quadrática
  - $O(N^3) \rightarrow$ cúbica
  - $O(2^N) \rightarrow$ exponencial
  - $O(N!) \rightarrow$ fatorial

Análise de Complexidade

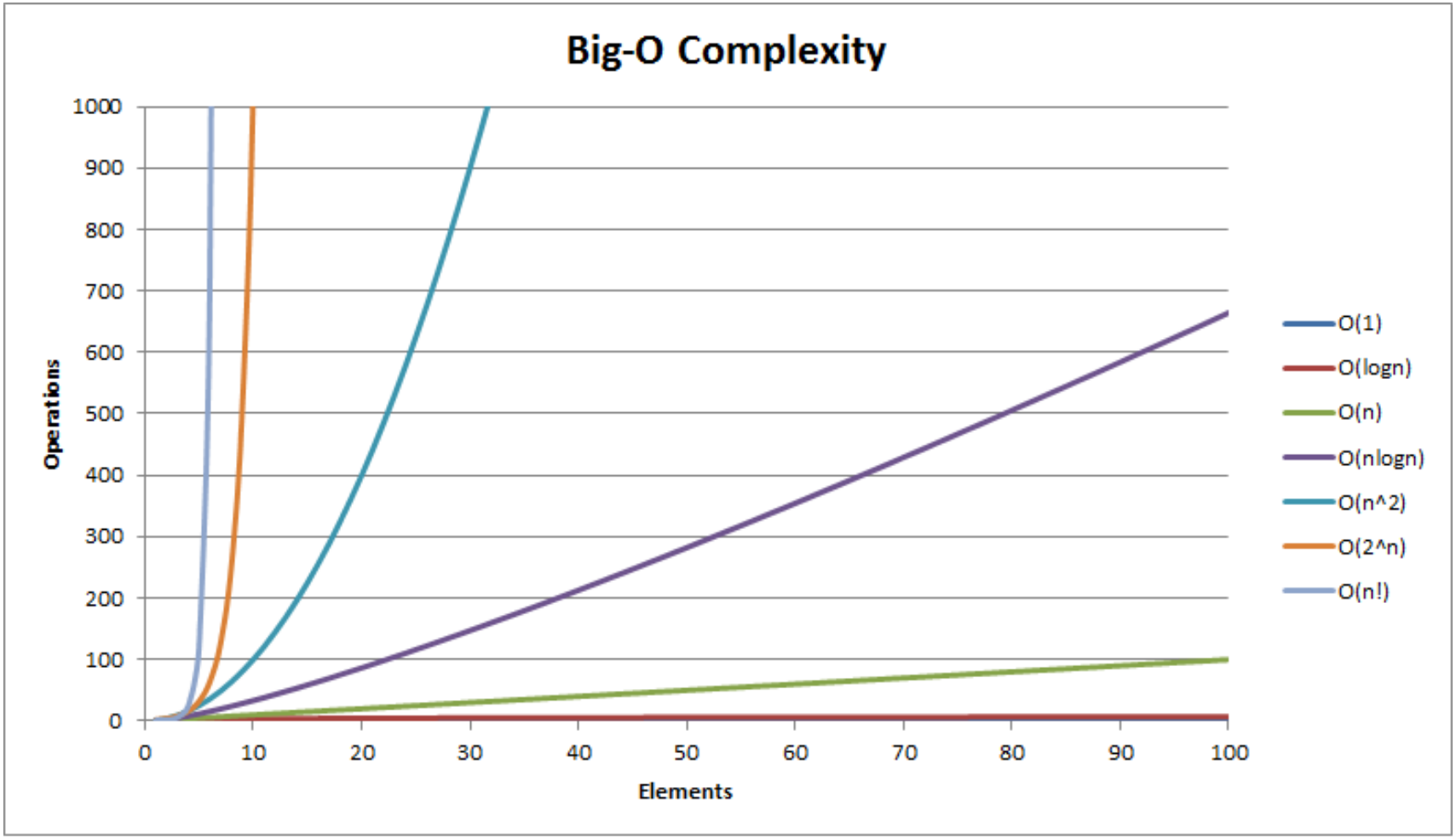


Figura: Ordens de grandeza.

Análise de Complexidade

- Exemplo: Analisar a solução iterativa de um algoritmo que leia um valor inteiro N, calcule e imprima o seu fatorial. Se N for negativo, imprimir uma mensagem de erro.

```
int n, c, fat = 1;
cout << "Digite n" << endl;
cin >> n;
if(n >= 0)
{
  for(c = 1; c<=n; c++)
    fat = fat * c;
  cout << "Fatorial = " << fat << endl;
}
else
  cout << "Valor negativo" << endl;
```

- O algoritmo será estudado para os seguintes casos:
  - $n < 0, n = 0, n = 1$ e $n > 1$

Análise de Complexidade

- Caso $n < 0$

```
int n, c, fat = 1;
cout << "Digite n" << endl;                    // 1
cin >> n;                                      // 1
if(n >= 0)                                     // 1
{
  for(c = 1; c<=n; c++)
    fat = fat * c;
  cout << "Fatorial = " << fat << endl;
}
else
  cout << "Valor negativo" << endl;            // 1
```

- Soma das frequências $= 4$

Análise de Complexidade

- Caso $n = 0$

```
int n, c, fat = 1;
cout << "Digite n" << endl;                    // 1
cin >> n;                                      // 1
if(n >= 0)                                     // 1
{
  for(c = 1; c<=n; c++)                        // 1
    fat = fat * c;
  cout << "Fatorial = " << fat << endl;        // 1
}
else
  cout << "Valor negativo" << endl;
```

- Soma das frequências $= 5$

Análise de Complexidade

- Caso $n = 1$

```
int n, c, fat = 1;
cout << "Digite n" << endl;                    // 1
cin >> n;                                      // 1
if(n >= 0)                                     // 1
{
  for(c = 1; c<=n; c++)                        // 2
    fat = fat * c;                             // 1
  cout << "Fatorial = " << fat << endl;        // 1
}
else
  cout << "Valor negativo" << endl;
```

- Soma das frequências $= 7$

Análise de Complexidade

- Caso $n = 1$

```
int n, c, fat = 1;
cout << "Digite n" << endl;                    // 1
cin >> n;                                      // 1
if(n >= 0)                                     // 1
{
  for(c = 1; c<=n; c++)                        // n+1
    fat = fat * c;                             // n
  cout << "Fatorial = " << fat << endl;        // 1
}
else
  cout << "Valor negativo" << endl;
```

- Soma das frequências $= 2n + 5$

Análise de Complexidade

Comportamento assintótico

- Para valores suficientemente pequenos de n, qualquer algoritmo custa pouco para ser executado, mesmo os ineficientes.
- Nesse caso, a escolha de um algoritmo não é um problema crítico.
- É importante analisar algoritmos para grandes valores de n
- Portanto, estuda-se o **comportamento assintótico** das funções de complexidade de um programa, isto é, o comportamento para grandes valores de n.

Análise de Complexidade

- De volta ao exemplo anterior do fatorial.
- No caso mais geral $n > 1$, a soma das frequências é de $2n + 5$.
- Como deseja-se estudar o comportamento apenas para n grande, pode-se desprezar as constantes e os termos de menor ordem.
- Assim, conclui-se que o programa possui complexidade linear, isto é, $O(n)$.

Análise de Complexidade

Exemplo - Algoritmo 1

- Analisar o tempo de processamento de um programa para calcular o seguinte somatório (série geométrica):

$$S = \sum_{i=0}^{n} x^i$$

```
float soma(int x, int n)
{
  int soma = 0;                    // 1
  for(int i=0; i<=n; i++)          // n+2
  {
    int prod = 1;                  // n+1
    for(int j=0; j<i; j++)
      prod = prod*x;               // ∑_{i=0}^{n} i
    soma = soma + prod;            // n+1
  }
  return soma;                     // 1
}
```

Análise de Complexidade

Exemplo - Algoritmo 1

- O tempo de processamento $T(n)$ desse programa é obtido somando-se a execução de todas instruções listadas anteriormente.

$$T(n) = 1 + (n+2) + (n+1) + (\sum_{i=0}^{n} i) + (n+1) + 1$$

$$T(n) = 6 + 3n + \sum_{i=0}^{n} i = 6 + 3n + \frac{n(n+1)}{2}$$

$$T(n) = \frac{n^2}{2} + \frac{7n}{2} + 6$$

- E assim, conclui-se que esse algoritmo é $O(n^2)$.

Análise de Complexidade

Exemplo - Algoritmo 2

- Pode-se utilizar um algoritmo conhecido como **algoritmo de Horner** para realizar esse cálculo.

$$
\begin{aligned}
S = \sum_{i=0}^{n} x^i &= 1 + x + x^2 + x^3 + \ldots + x^n \\
&= 1 + x(1 + x + x^2 + \ldots + x^{n-1}) \\
&= 1 + x(1 + x(1 + x + \ldots + x^{n-2})) \\
&= \ldots \\
&= 1 + x(1 + x(1 + x(1 + \ldots (1 + x(1 + x))) \ldots))
\end{aligned}
$$

Análise de Complexidade

Exemplo - Algoritmo 2

- Algoritmo de Horner

```
float somaHorner(int x, int n)
{
  int i, soma = 0;                    // 1
  for(i=0; i<=n; i++)                 // n+1
  {
    soma = soma*x + 1;                // n+2
  }
  return soma;                        // 1
}
```

- O tempo de processamento é $T(n) = 2n + 5$
- Portanto, esse algoritmo é $O(n)$.

Análise de Complexidade

Exemplo - Algoritmo 3

- Fórmula fechada

$$S = \sum_{i=0}^{n} x^i = \frac{x^{n+1} - 1}{x - 1}$$

- Para calcular a potência, usa-se `pot(int x, int n)` que possui complexidade $T(n) = \log_2 n + 2$

```
float soma(int x, int n)
{
  return (pot(x,n+1)-1)/(x-1);
}
```

- Portanto, esse algoritmo é $O(\log_2 n)$.

Análise de Complexidade

Exemplo

- Comparação dos três algoritmos
  - Algoritmo 1: $T(n) = \frac{n^2}{2} + \frac{7n}{2} + 6 \quad \Rightarrow \quad O(n^2)$
  - Algoritmo 2: $T(n) = 2n + 5 \quad \Rightarrow \quad O(n)$
  - Algoritmo 3: $T(n) = \log_2 n + 2 \quad \Rightarrow \quad O(\log_2 n)$

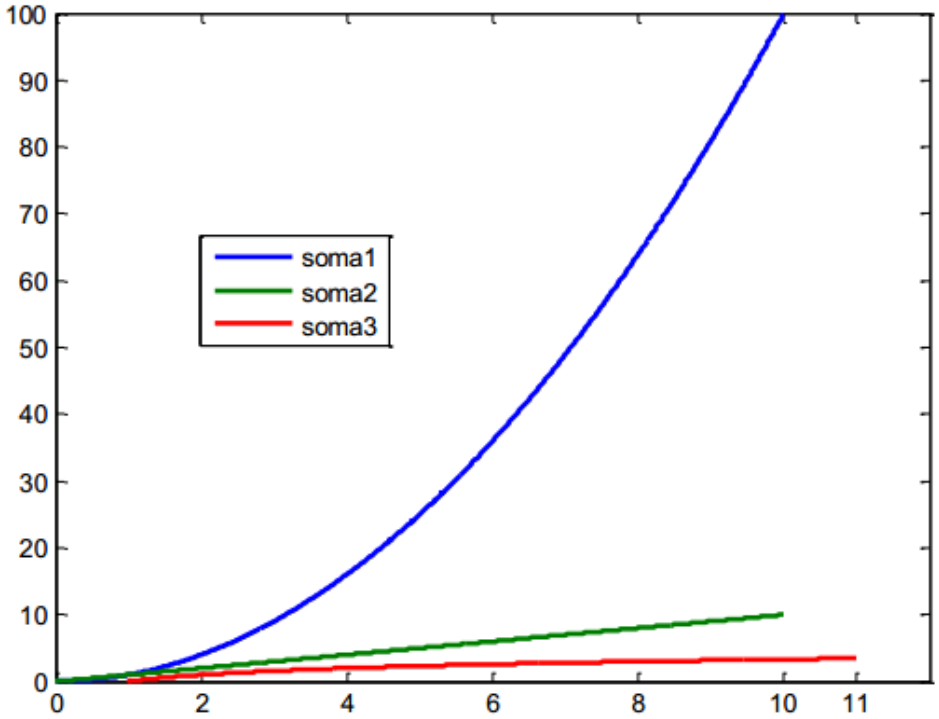


Figura: Comparação dos algoritmos.

Análise de Complexidade

- Tabela com tempos de execução.

FUNÇÃO DE COMPLEXIDADE	n (tamanho do problema)		
	20	40	60
n	0.0002 s	0.0004 s	0.0006 s
$n \log_2 n$	0.0009 s	0.0021 s	0.0035 s
n^2	0.0040 s	0.0160 s	0.0360 s
n^3	0.0800 s	0.6400 s	2.1600 s
2^n	10.0000 s	27 dias	3660 séculos
3^n	580 minutos	38550 séculos	$1.3*10^{14}$ séculos

- Algumas ordens de grandeza de complexidade tornam proibitivo a aplicabilidade do algoritmo, devendo ser usado apenas quando não se conheça solução de menor complexidade.

Análise de Complexidade

Exercícios

1. Calcular x^n e determinar a complexidade da sua solução.
2. Qual a complexidade do algoritmo abaixo?

```
int maior(int n, int v[])
{
  int m = v[0];
  for (int i=1; i<n; i++ )
  {
    if( v[i] >= m ) {
      m = v[i];
    }
  }
  return m;
}
```

Análise de Complexidade

Exercícios

3. Qual a complexidade das funções `f`, `g` e `h`?

```
int f(int n){
  int i, soma=0;
  for(i=1; i<=n; ++i)
    soma += 1;
  return soma;
}

int g(int n){
  int i, soma=0;
  for(i=1; i<=n; ++i)
    soma += i + f(i);
  return soma;
}

int h(int n){
  return f(n) + g(n);
}
```